

CH11 — Pointers with Structures

SLIDE 1 — What Is a Pointer to a Structure?

A **structure** can store different types of data together.

Example:

None

```
struct Student
{
    int age;
    float marks;
};
```

We can create a structure variable:

None

```
struct Student s;
```

A pointer can store the **address of a structure variable**.

None

```
struct Student *p = &s;
```

Easy Analogy

Think of a structure as a **student information box**:

None

Student



↑
p

★ Remember:

Structure → Stores data

Pointer → Stores address

SLIDE 2 — Accessing Structure Members Using a Pointer

Suppose:

None

```
struct Student
{
    int age;
};

struct Student s;

s.age = 20;

struct Student *p = &s;
```

A pointer can access the structure's member using:

None

`(*p).age`

This gives:

None

20

💡 **Easy Way to Remember:**

`p` → address of structure

`*p` → structure itself

`(*p).age` → access `age`

📄 SLIDE 3 — ★ The Arrow Operator ->

C provides an easier way to access structure members through a pointer.

Instead of:

None

`(*p).age`

we can write:

None

`p->age`

Both mean the same thing.

 Look at this:

None

`p → structure`

`p->age`

`p->marks`

`p->name`

Example:

None

`struct Student`

`{`

```
    int age;
};

struct Student s;

struct Student *p = &s;

p->age = 20;
```

💡 **Memory Trick:**

`p->member` → **ACCESS STRUCTURE MEMBER**

SLIDE 4 — Changing Structure Data Using a Pointer

A pointer can also **modify** structure members.

Example:

```
None
#include <stdio.h>

struct Student
{
    int age;
};

int main()
{
    struct Student s;
    struct Student *p = &s;

    p->age = 21;
```

```
    printf("%d", p->age);  
  
    return 0;  
}
```

Output:

```
None  
21
```

The pointer `p` points to `s`, so:

```
None  
p->age = 21;
```

changes the `age` member of the original structure.

Analogy:

The pointer has the **address of the student record**, so it can directly update the information inside it.

SLIDE 5 — RECAP

Pointers with Structures

Concept	Meaning
<code>struct Student s</code>	Creates a structure variable
<code>&s</code>	Gets structure's address
<code>struct Student *p</code>	Structure pointer
<code>*p</code>	Accesses the structure

`(*p).age` Accesses `age` using
pointer

`p->age` Easier way to access `age`

BIG MEMORY TRICK

STRUCTURE → ADDRESS → POINTER → `->` → MEMBER

None

`struct Student s`

↓

`&s`

↓

`p`

↓

`p->age`

↓

value

★ TAKEAWAY:

The `->` operator makes it easy to access **structure members using a pointer**.



QUESTIONS

1 What does `struct Student *p` create?

- A) A structure variable
- B) A pointer to a structure
- C) An integer variable
- D) A pointer to an integer

 **Hint:** `*` indicates that `p` is a pointer.

Glossary:

Structure pointer → A pointer that stores the address of a structure variable.

2) What is the output?

None

```
struct Student
{
    int age;
};

int main()
{
    struct Student s;
    struct Student *p = &s;

    p->age = 20;

    printf("%d", p->age);
}
```

- A) 0
- B) 10
- C) 20
- D) Address of s

 **Hint:** `p->age` accesses the `age` member through the pointer.

 **Glossary:**

Member → A variable stored inside a structure.

3) What does `p->age` mean?

- A) Access the `age` member through pointer `p`
- B) Create a new `age` variable
- C) Get the address of `p`
- D) Delete the structure

 **Hint:** `p` points to a structure.

 **Glossary:**

Access → Getting or changing the value of a variable.

4 Complete the Code

None

```
struct Student
{
    int age;
};

struct Student s;
struct Student *p = &s;

_____ = 21;
```

Change the `age` member using the structure pointer.

 **Hint:** Use `p` and the arrow operator.

 **Glossary:**

Modify → To change an existing value.

5 Which statement is equivalent to `p->age`?

- A) `*p.age`
- B) `(*p).age`
- C) `*p->age`
- D) `&p.age`

 **Hint:** The pointer is dereferenced first, then the member is accessed.

 **Glossary:**

Dereference → Accessing the structure stored at the address held by the pointer.

